

## Experiment-9

**Name:** - C. Jyothi

**Regno:** -192425312

**Course Code:** -MLA0305

**Slot:** B

---

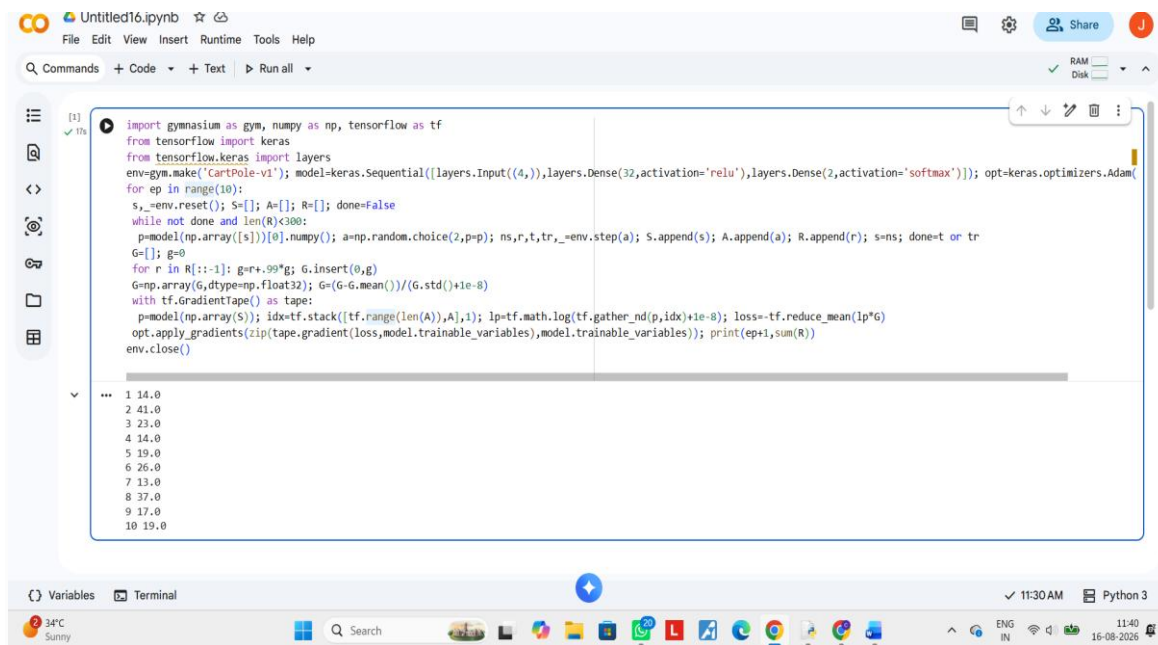
**Title:** Implementation of the REINFORCE Policy Gradient Algorithm

**Aim:**

To implement the REINFORCE algorithm and understand policy gradient methods for directly optimizing stochastic policies.

**Procedure:**

1. Select a suitable discrete-action environment.
2. Define a policy neural network.
3. Initialize policy parameters.
4. Generate complete episodes using the current policy.
5. Calculate the return for each time step.
6. Compute policy gradients using the REINFORCE objective.
7. Update the policy network using gradient ascent.
8. Repeat the process for multiple episodes.
9. Record episode rewards.
10. plot the learning curve.



The screenshot shows a Jupyter Notebook interface with a code cell containing Python code for training a REINFORCE policy on the CartPole-v1 environment. The code imports gymnasium, numpy, tensorflow, and keras. It defines a neural network with two hidden layers (4 and 32 units) and a softmax output layer. The training loop runs for 10 episodes, each with a maximum of 300 steps. It calculates the cumulative reward and the log-probability of the actions taken, then applies gradients to update the model parameters using the Adam optimizer. The results of the training are displayed in the output cell as a list of cumulative rewards for each episode.

```
import gymnasium as gym, numpy as np, tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras import Sequential([layers.Input((4,)), layers.Dense(32, activation='relu'), layers.Dense(2, activation='softmax')]); opt=keras.optimizers.Adam
env=gym.make('CartPole-v1'); model=model; opt=opt
for ep in range(10):
    s,_=env.reset(); S=[]; A=[]; R=[]; done=False
    while not done and len(R)<300:
        p=model(np.array([s]))[0].numpy(); a=np.random.choice(2,p=p); ns,r,t,tr,_=env.step(a); S.append(s); A.append(a); R.append(r); s=ns; done=t or tr
    G=[]; g=0
    for r in R[::-1]: g=r+.99*g; G.insert(0,g)
    G=np.array(G,dtype=np.float32); G=(G-G.mean())/(G.std()+1e-8)
    with tf.GradientTape() as tape:
        p=model(np.array(S)); idx=tf.stack([tf.range(len(A)),A],1); lp=tf.math.log(tf.gather_nd(p,idx)+1e-8); loss=-tf.reduce_mean(lp*G)
    opt.apply_gradients(zip(tape.gradient(loss,model.trainable_variables),model.trainable_variables)); print(ep+1,sum(R))
env.close()
```

```
1 14.0
2 41.0
3 23.0
4 14.0
5 19.0
6 26.0
7 13.0
8 37.0
9 17.0
10 19.0
```

## Result:

The REINFORCE algorithm successfully learned a policy through direct policy optimization. The cumulative reward improved as training progressed.